Priority Queues, Ordered Sets & Maps

O(log n) structures that keep your data sorted while it changes

Learning goals

- Use `priority_queue` as a max-heap and a **min-heap**
- Know what a binary heap does and costs
- Use `set`, `multiset`, `map` with `lower_bound` / `upper_bound`
- Erase **one** copy from a `multiset` (classic bug!)
- Choose between `map` and `unordered_map`
- Use the **policy-based ordered set** for k-th element and rank queries
- Recognise "greedy + heap" and "sweep + set" patterns

Motivation: CSES 1091 — Concert Tickets

n tickets with prices; m customers arrive in order, each with a max price t.

Each buys the **most expensive ticket ≤ t** (it disappears), or gets −1.

```
tickets: 5 3 7 8 5        customers: 4 8 3
customer 4 → 3   (left: 5 5 7 8)
customer 8 → 8   (left: 5 5 7)
customer 3 → -1
```

Sorted array + binary search: but **deleting** is O(n). ×

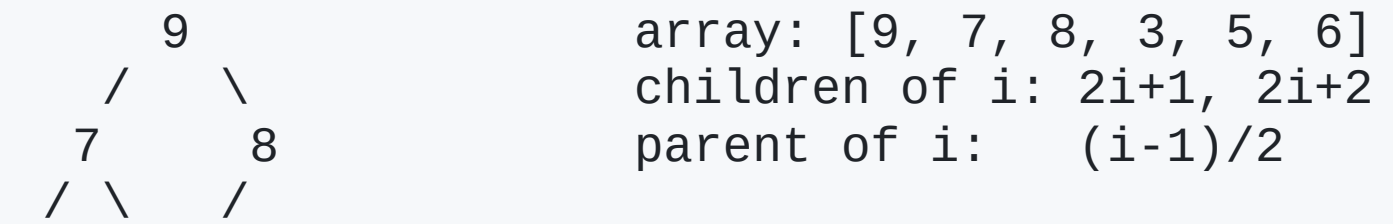
Need: sorted + fast insert/erase + "largest ≤ t" → **multiset**, O(log n).

The toolbox at a glance

Structure	Keeps	Main ops	Cost
`priority_queue`	only the max accessible	push, top, pop	O(log n)
`set`	sorted unique keys	insert, erase, find, bounds	O(log n)
`multiset`	sorted keys with repeats	same	O(log n)
`map`	sorted key → value	`[]`, find, bounds	O(log n)
`unordered_map`	key → value, no order	`[]`, find	O(1) avg

Binary heap: the idea

Complete binary tree in an array; **parent ≥ children**.



- push: append at the end, **sift up** — O(log n)
- pop: move last to root, **sift down** — O(log n)
- top: root — O(1)
- build from n items: O(n)

`priority_queue` in C++

```
priority_queue<int> pq;                    // MAX-heap by default
pq.push(5); pq.push(1); pq.push(8);
pq.top();   // 8
pq.pop();   // removes 8 (returns void)

// MIN-heap:
priority_queue<int, vector<int>, greater<int>> mn;

// pairs compare lexicographically: (dist, node)
priority_queue<pair<long long,int>, vector<pair<long long,int>>,
               greater<>> pq2;
```

No iteration, no erase of arbitrary elements, no decrease-key.

Trick: min-heap by negation

```
priority_queue<long long> pq;
pq.push(-x);                 // store negated
long long smallest = -pq.top();
```

Short, but easy to forget a minus sign somewhere.

`greater<>` is safer for pairs and structs.

Custom order with a lambda:

```
auto cmp = [](const Job& a, const Job& b) { return a.deadline > b.deadline; };
priority_queue<Job, vector<Job>, decltype(cmp)> pq(cmp);
```



⚠️ Comparator means "a has **lower** priority than b".

Heap pattern: repeatedly take the best

- **Merge k sorted lists** / Huffman-style "combine two smallest"
- **Scheduling**: assign each task to the machine that frees up first
- **Dijkstra** (Week 17): extract the closest unfinished vertex
- **Top-k** of a stream: keep a min-heap of size k

Lazy deletion: can't erase from the middle? Push updated copies and **skip stale entries** when they reach the top.

Worked example: CSES 1164 — Room Allocation

Customers [a, b] (days). Minimum rooms; print a room per customer.

1. Sort customers by arrival
2. Min-heap of (departure day, room id) of occupied rooms
3. If the earliest departure < a → reuse that room; else open a new one

```
(1,2) → room 1           heap: (2,1)
(2,4) → 2 ≥ 2, new room 2   heap: (2,1) (4,2)
(4,4) → 2 < 4, reuse room 1  heap: (4,1) (4,2)
answer: 2 rooms, assignment 1 2 1
```

O(n log n).

`set` and `multiset`

Balanced BST (red-black tree), iterated in **sorted order**.

```
set<int> s = {5, 1, 8};
s.insert(3);              // {1, 3, 5, 8}
s.count(5);               // 1
s.erase(5);               // by value
*s.begin();               // min = 1
*s.rbegin();              // max = 8
for (int x : s) ...       // 1 3 8
```

`multiset` allows duplicates: `{5, 5, 7}`.

`lower_bound` / `upper_bound`

```
s = {1, 3, 3, 5, 8}  (multiset)
s.lower_bound(3)  → first element ≥ 3    → the first 3
s.upper_bound(3)  → first element > 3    → 5
s.lower_bound(9)  → s.end()
```

Want	Code
smallest ≥ x	`it = s.lower_bound(x)` (check `!= end`)
smallest > x	`s.upper_bound(x)`
largest ≤ x	`it = s.upper_bound(x); if (it != begin) --it;`
largest < x	`it = s.lower_bound(x); if (it != begin) --it;`



⚠️ Use the **member** function: `std::lower_bound(s.begin(), s.end(), x)` is **O(n)**!

The multiset erase trap

```
multiset<int> ms = {5, 5, 7};
ms.erase(5);                // removes ALL copies → {7}   ✗
ms.erase(ms.find(5));       // removes ONE copy  → {5, 7} ✓
```

`erase(ms.find(x))` when x is absent → erasing `end()` → **crash**.

Check `find(x) != end()` first.

Concert Tickets solution

```
multiset<int> tickets(h.begin(), h.end());
for (int t : customers) {
    auto it = tickets.upper_bound(t);    // first price > t
    if (it == tickets.begin()) { cout << -1 << '\n'; continue; }
    --it;                                // largest price ≤ t
    cout << *it << '\n';
    tickets.erase(it);                   // erase this ONE ticket
}
```

O((n + m) log n).

`map` : sorted dictionary

```
map<string, int> freq;
for (auto& w : words) freq[w]++;           // [] inserts 0 if missing
for (auto& [w, c] : freq) cout << w << ' ' << c;   // sorted by key

if (freq.count("x")) ...                   // test WITHOUT inserting
auto it = freq.lower_bound("m");           // bounds work on keys
freq.erase("b");
```



`freq[key]` in a lookup **inserts** a default value — changes `size()` and may break iteration.

map vs unordered_map

	`map`	`unordered_map`
order	sorted	none
lookup	O(log n)	O(1) average, O(n) worst
bounds / min / max	yes	no
key types	anything with `<`	needs a hash

- Default to `map` unless profiling says otherwise
- `unordered_map` can be **hacked** on Codeforces (anti-hash tests)
  → use a custom randomized hash, or `.reserve()` + avoid it in hacks
- Values ≤ 10^6? A plain **array** beats both

Sweep with a set: CSES 1163 Traffic Lights

Street [0, x]; add lights one by one; after each, print longest unlit segment.

- `set<int> pos = {0, x}`; `multiset<int> len = {x}`
- Add p: find neighbours a < p < b with `upper_bound`
- `len`: erase one (b − a), insert (p − a) and (b − p)
- Answer: `*len.rbegin()`

Two coordinated sorted structures: O(n log n).

Policy-based data structures (GNU)

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
                         tree_order_statistics_node_update>;
```

A `set` that also knows **ranks**:

- `s.find_by_order(k)` → iterator to the k-th smallest (0-indexed)
- `s.order_of_key(x)` → number of elements **< x**

Both O(log n). GCC only (all major judges use GCC).

Ordered multiset trick

`ordered_set<int>` rejects duplicates. Store **pairs** (value, unique id):

```
ordered_set<pair<int,int>> s;
s.insert({x, id++});
s.order_of_key({x, INT_MIN});   // # elements < x
s.order_of_key({x, INT_MAX});   // # elements ≤ x
s.erase(s.lower_bound({x, INT_MIN}));   // erase one copy of x
s.find_by_order(s.size() / 2)->first;   // median
```

(`less_equal<T>` as comparator "works" but breaks `erase` / `find` — avoid.)

Example: CSES 2162 — Josephus Problem II

n children in a circle; repeatedly skip k and remove the next. Order?

n, k ≤ $2 \cdot 10^5$ → simulation with a list is O(nk). ×

```
ordered_set<int> s;   // 1..n
int pos = 0;
while (!s.empty()) {
    pos = (pos + k) % s.size();
    auto it = s.find_by_order(pos);
    cout << *it << ' ';
    s.erase(it);       // next child slides into index pos
}
```

n = 7, k = 2 → `3 6 2 7 5 1 4`. O(n log n).

Choosing the right structure

Need	Use
repeatedly extract min/max	`priority_queue`
+ delete arbitrary / see both ends	`multiset`
nearest value ≥ / ≤ x, with updates	`set` / `multiset` + bounds
counts / attributes per key	`map` (or array if keys small)
k-th smallest or rank, with updates	PBDS ordered_set
static data, no updates	sorted `vector` + binary search

Complexity summary

Operation	pq	set/map	unordered_map	ordered_set
insert	log n	log n	1 avg	log n
erase	top only, log n	log n	1 avg	log n
min / max	1 (one side)	1	—	1
bounds	—	log n	—	log n
k-th / rank	—	—	—	log n

Constant factors: trees are ~5–10× slower than arrays. 10^6 ops ≈ fine.

Pitfalls

- `priority_queue` is a **max**-heap; comparator direction is inverted
- `multiset::erase(value)` deletes **all** copies
- `std::lower_bound` on set iterators is O(n) — use the member
- Dereferencing `end()` / decrementing `begin()`
- `map[key]` silently inserts
- Modifying a set element through an iterator — not allowed (const)
- Iterator invalidation: erasing the element you're standing on
  → `it = s.erase(it);`

Practice problems

- CSES 1621 — Distinct Numbers · CSES 1640 — Sum of Two Values
- CSES 1091 — Concert Tickets
- CSES 1164 — Room Allocation
- CSES 1163 — Traffic Lights
- CSES 2162 — Josephus Problem II
- CSES 1076 — Sliding Window Median · CSES 1144 — Salary Queries
- Codeforces 4C — Registration System
- AtCoder ABC 141 D — Powerful Discount Tickets · ABC 217 D — Cutting Woods
- Codeforces 1526C2 — Potions (Hard Version)

Summary

- Heap: fast access to **one** extreme; great for greedy & Dijkstra
- set/multiset/map: sorted, dynamic, **bounds** queries in O(log n)
- Erase one multiset copy via iterator; use member `lower_bound`
- PBDS ordered_set adds k-th element and rank
- Pick the simplest structure that supports every operation you need
- Next week: **Dynamic Programming I**